

CSE340 Final Theory Master Guide

Chapters 2, 3, 4 & 5

Theory-first exam preparation based on the supplied course resources

Faculty suggestion:

“A small suggestion, make sure you clearly understand all the theoretical concepts covered in each chapter.”

Prepared from the conversation and supplied lecture resources

Contents

Importance Key	4
1 CHAPTER 2 — INSTRUCTIONS: LANGUAGE OF THE COMPUTER	4
1.1 2.1 What is an Instruction Set? — VERY IMPORTANT	4
1.2 2.2 Design Principles	5
1.3 2.3 Operands — VERY IMPORTANT	5
1.4 2.4 RISC-V Registers — VERY IMPORTANT	5
1.5 2.5 Register vs Memory — VERY IMPORTANT	6
1.6 2.6 Memory Operands — VERY IMPORTANT	6
1.7 2.7 Signed and Unsigned Integers — VERY IMPORTANT	6
1.8 2.8 Two's Complement — VERY IMPORTANT	6
1.9 2.9 Instruction Representation — VERY IMPORTANT	7
1.10 2.10 R-Format — VERY IMPORTANT	7
1.11 2.11 I-Format — VERY IMPORTANT	7
1.12 2.12 S-Format — VERY IMPORTANT	7
1.13 2.13 Branch Instructions — VERY IMPORTANT	7
1.14 2.14 Basic Blocks — IMPORTANT	8
1.15 2.15 Logical / Bitwise Operations — VERY IMPORTANT	8
1.16 2.16 Shift Operations — VERY IMPORTANT	9
1.17 2.17 Wide Immediates — IMPORTANT	9
1.18 2.18 Branch Addressing — VERY IMPORTANT	9
1.19 2.19 Jump Addressing — IMPORTANT	9
1.20 2.20 Procedures / Functions — VERY IMPORTANT	10
1.21 2.21 Caller/Callee Register Convention — VERY IMPORTANT	10
1.22 2.22 Stack — IMPORTANT	10
1.23 2.23 Arrays vs Pointers — IMPORTANT	10
1.24 2.24 Synchronization — VERY IMPORTANT	11
1.25 2.25 Stored-Program Concept — IMPORTANT	11
1.26 2.26 Compiler, Assembler, Linker, Loader — VERY IMPORTANT	11
1.27 2.27 Object Module — IMPORTANT	11
1.28 2.28 Static vs Dynamic Linking — IMPORTANT	11
2 CHAPTER 3 — ARITHMETIC FOR COMPUTERS	12
2.1 3.1 Integer Addition — VERY IMPORTANT	12
2.2 3.2 Integer Subtraction — VERY IMPORTANT	12
2.3 3.3 Multiplication — VERY IMPORTANT	12
2.4 3.4 Multiplication Hardware — IMPORTANT	12
2.5 3.5 RISC-V Multiplication — VERY IMPORTANT	12
2.6 3.6 Integer Division — KNOW (resource limitation)	12
2.7 3.7 Floating Point — VERY IMPORTANT	13
2.8 3.8 Why Floating Point?	13
2.9 3.9 IEEE-754 — EXTREMELY IMPORTANT IMPORTANT	13
2.10 3.10 Sign Bit — VERY IMPORTANT	13
2.11 3.11 Exponent Bias — VERY IMPORTANT	13
2.12 3.12 Fraction / Significand — VERY IMPORTANT	14
2.13 3.13 Floating-Point Precision — IMPORTANT	14
2.14 3.14 Special Floating-Point Values — VERY IMPORTANT	14
2.15 3.15 Decimal to IEEE-754 — VERY IMPORTANT	14
2.16 3.16 IEEE-754 to Decimal — VERY IMPORTANT	14
2.17 3.17 Floating-Point Addition/Subtraction — EXTREMELY IMPORTANT IMPORTANT	15
2.18 3.18 Floating-Point Multiplication — VERY IMPORTANT	15

2.19	3.19 Floating-Point Arithmetic Hardware — IMPORTANT	15
2.20	3.20 RISC-V Floating-Point Registers — IMPORTANT	15
2.21	3.21 RISC-V Floating-Point Instructions — IMPORTANT	16
2.22	3.22 Big Theory of Chapter 3 — VERY IMPORTANT	16
3	CHAPTER 4 — THE PROCESSOR	17
3.1	4.1 CPU Performance — VERY IMPORTANT	17
3.2	4.2 Instruction Execution — VERY IMPORTANT	17
3.3	4.3 Datapath — VERY IMPORTANT	17
3.4	4.4 Control Unit — VERY IMPORTANT	17
3.5	4.5 Combinational vs Sequential Logic — VERY IMPORTANT	17
3.6	4.6 Multiplexer — IMPORTANT	18
3.7	4.7 Register — IMPORTANT	18
3.8	4.8 Clocking Methodology — VERY IMPORTANT	18
3.9	4.9 Instruction Fetch — IMPORTANT	18
3.10	4.10 R-Type Datapath — VERY IMPORTANT	18
3.11	4.11 Load/Store Datapath — VERY IMPORTANT	18
3.12	4.12 Branch Datapath — VERY IMPORTANT	19
3.13	4.13 Why Separate Instruction and Data Memory? — VERY IMPORTANT	19
3.14	4.14 ALU Control — EXTREMELY IMPORTANT	19
3.15	4.15 Why Does ld Use ADD in the ALU?	19
3.16	4.16 Single-Cycle Processor — VERY IMPORTANT	19
3.17	4.17 Critical Path — EXTREMELY IMPORTANT	20
3.18	4.18 Pipelining — EXTREMELY IMPORTANT	20
3.19	4.19 Five Pipeline Stages — EXTREMELY IMPORTANT	20
3.20	4.20 Why Pipelining Improves Performance — VERY IMPORTANT	20
3.21	4.21 Why RISC-V Is Pipeline-Friendly — IMPORTANT	20
3.22	4.22 Pipeline Registers — IMPORTANT	21
3.23	4.23 Pipeline Hazards — EXTREMELY IMPORTANT	21
3.24	4.24 Structural Hazard — VERY IMPORTANT	21
3.25	4.25 Data Hazard — EXTREMELY IMPORTANT	21
3.26	4.26 Forwarding / Bypassing — EXTREMELY IMPORTANT	21
3.27	4.27 Load-Use Hazard — EXTREMELY IMPORTANT	21
3.28	4.28 Code Scheduling — IMPORTANT	22
3.29	4.29 Control Hazard — VERY IMPORTANT	22
4	CHAPTER 5 — LARGE AND FAST: MEMORY HIERARCHY	23
4.1	5.1 Principle of Locality — EXTREMELY IMPORTANT	23
4.2	5.2 Memory Hierarchy — EXTREMELY IMPORTANT	23
4.3	5.3 SRAM vs DRAM — IMPORTANT	23
4.4	5.4 Cache — EXTREMELY IMPORTANT	23
4.5	5.5 Block / Cache Line — VERY IMPORTANT	24
4.6	5.6 Hit, Miss, Hit Ratio, Miss Ratio — EXTREMELY IMPORTANT	24
4.7	5.7 Cache Organizations — EXTREMELY IMPORTANT	24
4.8	5.8 Direct-Mapped Cache — EXTREMELY IMPORTANT	24
4.9	5.9 Tag and Valid Bit — EXTREMELY IMPORTANT	24
4.10	5.10 Address Division — EXTREMELY IMPORTANT	25
4.11	5.11 How to Determine Hit or Miss — EXTREMELY IMPORTANT	25
4.12	5.12 Fully Associative Cache — IMPORTANT	25
4.13	5.13 Set-Associative Cache — IMPORTANT	25
4.14	5.14 Block Size — VERY IMPORTANT	25
4.15	5.15 Cache Pollution — IMPORTANT	26
4.16	5.16 Early Restart — IMPORTANT	26
4.17	5.17 Critical-Word-First — IMPORTANT	26

4.18	5.18	What Happens During a Cache Miss? — VERY IMPORTANT	26
4.19	5.19	Write-Through — VERY IMPORTANT	26
4.20	5.20	Write-Back — VERY IMPORTANT	26
4.21	5.21	Write Allocation — IMPORTANT	26
4.22	5.22	AMAT — EXTREMELY IMPORTANT IMPORTANT	27
4.23	5.23	Multilevel Cache — IMPORTANT	27
4.24	5.24	Virtual Memory — EXTREMELY IMPORTANT IMPORTANT	27
4.25	5.25	Why Virtual Memory? — VERY IMPORTANT	27
4.26	5.26	Pages — IMPORTANT	27
4.27	5.27	Page Fault — EXTREMELY IMPORTANT IMPORTANT	27
4.28	5.28	Page Table — EXTREMELY IMPORTANT IMPORTANT	28
4.29	5.29	Reference Bit — IMPORTANT	28
4.30	5.30	Dirty Bit — VERY IMPORTANT	28
4.31	5.31	TLB — EXTREMELY IMPORTANT IMPORTANT	28
4.32	5.32	TLB Miss vs Page Fault — EXTREMELY IMPORTANT IMPORTANT	28
4.33	5.33	TLB Miss Handler — IMPORTANT	28
4.34	5.34	TLB and Cache Interaction — IMPORTANT	29
4.35	5.35	Memory Protection — VERY IMPORTANT	29
4.36	5.36	Common Framework for Memory Hierarchy — EXTREMELY IMPORTANT IMPORTANT	29
5		THE BIG CONCEPTUAL CONNECTIONS	30
5.1		Connection 1: Why do caches work? — VERY IMPORTANT	30
5.2		Connection 2: Why does pipelining improve performance but create hazards? — VERY IMPORTANT	30
5.3		Connection 3: Why is a single-cycle processor inefficient? — VERY IMPORTANT	30
5.4		Connection 4: Why is RISC-V designed with regular instruction formats? — VERY IMPORTANT	30
5.5		Connection 5: Why does virtual memory behave like cache? — VERY IMPORTANT	30
5.6		Connection 6: Cache and Virtual Memory Share the Same Framework — EXTREMELY IMPORTANT IMPORTANT	31
6		TOP THEORY TOPICS TO PRIORITIZE	32
7		SELF-TEST: QUESTIONS YOU SHOULD BE ABLE TO ANSWER WITHOUT NOTES	33
8		FINAL-NIGHT PRIORITY ORDER	35
9		THE FOUR CHAPTERS AS ONE BIG STORY	35

Importance Key

- **VERY IMPORTANT** – high-priority theory; understand it well enough to explain it in a written answer.
- **IMPORTANT** – important; likely useful for conceptual/written questions.
- **KNOW** – understand the basic idea and terminology.
- Lower-priority material is still included where relevant to the supplied resources.

1 CHAPTER 2 — INSTRUCTIONS: LANGUAGE OF THE COMPUTER

The chapter is fundamentally about how software communicates with hardware through instructions. It covers instruction sets, operands, instruction representation, logical operations, decisions, procedures, synchronization, translation/loading, arrays and pointers, and comparisons with MIPS and x86.

1.1 2.1 What is an Instruction Set? — **VERY IMPORTANT**

An **instruction** is a command given to the computer's hardware.

An **instruction set** is the complete repertoire of instructions that a processor understands.

Examples:

```
add
sub
ld
sd
beq
```

What is ISA?

ISA = Instruction Set Architecture.

Think of ISA as the **contract/interface between software and hardware**. It specifies:

- what instructions exist,
- what registers exist,
- how memory is accessed,
- how instructions are encoded,
- what operations instructions perform.

RISC-V

RISC stands for **Reduced Instruction Set Computer**. The supplied handout describes RISC-V as having a modular design:

Base ISA + Optional Extensions

The supplied resource defines:

Byte = 8 bits, Halfword = 16 bits, Word = 32 bits, Doubleword = 64 bits.

It also describes a RISC-V register file as 32×64 -bit registers.

Exam question: What is an ISA?

An Instruction Set Architecture is the interface between software and hardware that defines the instructions, registers, memory-access mechanisms, instruction formats, and programmer-visible behavior of a processor.

1.2 2.2 Design Principles

Principle 1: Simplicity favors regularity — **VERY IMPORTANT**

Arithmetic instructions generally use a consistent structure:

```
add destination, source1, source2
```

Regularity makes implementation simpler and can enable higher performance at lower cost.

Principle 2: Smaller is faster — **VERY IMPORTANT**

Registers are much smaller in number than main memory, but much faster. Frequently used data is preferably kept in registers.

Principle 3: Good design demands good compromises — **VERY IMPORTANT**

Different instruction formats complicate decoding somewhat, but allow instructions to remain uniformly 32 bits. The formats are kept as similar as practical.

1.3 2.3 Operands — **VERY IMPORTANT**

An **operand** is the part of an instruction containing the data being operated on or the location of that data.

Three operand types:

1. **Register operand** — data is inside a register.
2. **Memory operand** — data is stored in main memory.
3. **Immediate/constant operand** — the constant is contained directly in the instruction.

Example:

```
addi x22, x22, 4
```

The 4 is an immediate.

1.4 2.4 RISC-V Registers — **VERY IMPORTANT**

RISC-V has:

32 general-purpose registers
x0 - x31

Register	Purpose
x0	constant 0
x1	return address
x2	stack pointer
x3	global pointer
x4	thread pointer
x5–x7, x28–x31	temporaries
x8	frame pointer
x9, x18–x27	saved registers
x10–x11	function arguments/results
x12–x17	function arguments

These are programmer/compiler conventions rather than a statement that hardware intrinsically forces every register to have that semantic role.

1.5 2.5 Register vs Memory — **VERY IMPORTANT**

Registers provide very fast access but limited capacity. Memory provides large capacity but longer access time.

General flow:

```
Memory -> Load -> Register
Register -> Arithmetic
Register -> Store -> Memory
```

1.6 2.6 Memory Operands — **VERY IMPORTANT**

The supplied materials emphasize:

- memory is used for arrays, structures, and dynamic data,
- arithmetic generally requires values to be loaded into registers,
- results are stored back to memory,
- memory is byte addressed,
- RISC-V is presented as little-endian in the supplied material.

Little Endian

The least significant byte is stored at the lowest memory address.

Memory trick:

Little endian $\rightarrow$ little byte first.

1.7 2.7 Signed and Unsigned Integers — **VERY IMPORTANT**

For an unsigned n -bit number:

$$0 \leq x \leq 2^n - 1.$$

For an n -bit two's-complement signed number:

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

The sign bit is the most significant bit:

$$0 \rightarrow \text{non-negative}, \quad 1 \rightarrow \text{negative}.$$

1.8 2.8 Two's Complement — **VERY IMPORTANT**

To represent $-x$:

1. write the binary representation of x ,
2. invert all bits,
3. add 1.

Example:

$$+6 = 0110$$

$$0110 \rightarrow 1001 \rightarrow 1010$$

Therefore $-6 = 1010$ in 4-bit two's complement.

Why two's complement? It allows subtraction to be performed using addition hardware:

$$A - B = A + (-B).$$

1.9 2.9 Instruction Representation — **VERY IMPORTANT**

Instructions are represented as binary machine code. RISC-V uses 32-bit instruction words and a small number of regular formats.

Conceptually:

Assembly instruction → Machine code → Hardware decodes it.

1.10 2.10 R-Format — **VERY IMPORTANT**

R-format is used primarily for register-to-register arithmetic/logical operations.

Fields:

funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7 bits

Meaning:

- opcode — broad operation class,
- rd — destination register,
- rs1 — first source register,
- rs2 — second source register,
- funct3 — additional operation information,
- funct7 — further operation information.

Example:

`add x9, x20, x21`

means:

$$x9 = x20 + x21.$$

1.11 2.11 I-Format — **VERY IMPORTANT**

I-format is used for immediate arithmetic and loads.

The immediate is two's complement and sign-extended.

Conceptually:

immediate	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------

The immediate can be used as:

- a constant operand,
- a memory offset.

1.12 2.12 S-Format — **VERY IMPORTANT**

S-format is used for stores.

- *rs1* = base-address register,
- *rs2* = source-data register,
- immediate = offset.

The immediate is split into two pieces so that register fields remain in consistent positions.

1.13 2.13 Branch Instructions — **VERY IMPORTANT**

Examples:

```

beq rs1, rs2, label
bne rs1, rs2, label
blt rs1, rs2, label
bge rs1, rs2, label
bltu rs1, rs2, label
bgeu rs1, rs2, label

```

Meaning:

beq	equal
bne	not equal
blt	signed <
bge	signed $\geq$
bltu	unsigned <
bgeu	unsigned $\geq$

Signed vs Unsigned Comparison

The same bit pattern may have radically different meaning depending on interpretation.

For example:

11111111 ... 1111

can be -1 as signed but a very large positive number as unsigned.

Thus:

signed: $-1 < +1$

while:

unsigned: $2^n - 1 > 1$.

1.14 2.14 Basic Blocks — IMPORTANT

A **basic block** is a sequence of instructions with:

- no embedded branches except possibly at the end,
- no branch targets except at the beginning.

Compilers identify basic blocks for optimization.

Easy memory trick:

One entry, one exit.

1.15 2.15 Logical / Bitwise Operations — VERY IMPORTANT

The supplied slides cover:

AND

OR

XOR

Shift left

Shift right

They operate bit-by-bit.

AND

Useful for masking bits.

A mask bit of 1 tends to preserve the corresponding input bit; a mask bit of 0 clears it.

OR

Useful for setting bits.

XOR

Useful for bit-difference and related bit manipulation.

1.16 2.16 Shift Operations — VERY IMPORTANT**Logical left shift:**

`slli`

Moves bits left and inserts zero bits on the right.

For unsigned values:

$$x \ll i = x \cdot 2^i.$$

Logical right shift:

`srlr`

Moves bits right and inserts zeroes on the left.

For unsigned values:

$$x \gg i = \left\lfloor \frac{x}{2^i} \right\rfloor.$$

For an array of 64-bit values:

$$\text{address}(A[i]) = \text{base} + i \times 8.$$

Multiplication by 8 can be replaced with a left shift by 3.

1.17 2.17 Wide Immediates — IMPORTANT

Most constants fit in a 12-bit immediate. For larger constants, the material uses `lui` together with other instructions such as `addi`.

Conceptually:

$$\text{upper portion} \xrightarrow{\text{lui}} \text{combine lower portion} \xrightarrow{\text{addi}} \text{complete constant}.$$

1.18 2.18 Branch Addressing — VERY IMPORTANT

Branches use PC-relative addressing in the supplied material:

$$\text{Target} = PC + \text{immediate} \times 2.$$

Branches are usually nearby, so a compact PC-relative representation is sufficient.

1.19 2.19 Jump Addressing — IMPORTANT

`jal` uses a larger immediate than a conditional branch.

For larger addresses, the supplied material uses the general idea:

$$\text{lui} \rightarrow \text{upper address bits}, \quad \text{jalr} \rightarrow \text{lower portion} + \text{jump}.$$

1.20 2.20 Procedures / Functions — **VERY IMPORTANT**

A procedure call conceptually involves:

1. place arguments in registers,
2. transfer control to the procedure,
3. acquire storage if needed,
4. perform the procedure,
5. place the result in the return register,
6. return to the caller.

The supplied slides use $x10$ – $x17$ for arguments and $x1$ as the return-address register.

jal

`jal x1, ProcedureLabel`

Conceptually:

$x1 \leftarrow$ address of next instruction, $PC \leftarrow$ target.

jalr

Procedure return is shown as:

`jalr x0, 0(x1)`

Using $x0$ as destination discards the link because $x0$ cannot be changed.

1.21 2.21 Caller/Callee Register Convention — **VERY IMPORTANT**

Temporary registers:

$x5$ – $x7$, $x28$ – $x31$

are not preserved by the callee.

Saved registers:

$x8$ – $x9$, $x18$ – $x27$

must be preserved if used by the callee.

1.22 2.22 Stack — **IMPORTANT**

$x2$ is the stack pointer (**sp**).

The stack is used to store saved registers and procedure-local information.

General pattern:

```
save register -> stack
perform work
restore register <- stack
```

1.23 2.23 Arrays vs Pointers — **IMPORTANT**

Array indexing conceptually performs:

$$\text{address} = \text{base} + \text{index} \times \text{element size}.$$

Pointers already represent addresses, so pointer traversal can avoid repeated index calculations.

The supplied slides discuss **strength reduction**: multiplication by element size can become a shift, and pointer increments can eliminate repeated index calculations.

1.24 2.24 Synchronization — VERY IMPORTANT

If two processors share memory, unsynchronized access can cause a **data race**. The result can depend on the order of accesses.

An **atomic** operation prevents another access from interfering between the read and write.

lr.d

Load reserved:

lr.d rd, (rs1)

Loads memory and places a reservation on the address.

sc.d

Store conditional:

sc.d rd, (rs1), rs2

Succeeds only if the location was not changed since the reservation.

In the supplied material:

$$\text{success} \Rightarrow rd = 0, \quad \text{failure} \Rightarrow rd \neq 0.$$

These are used to implement synchronization primitives such as locks.

1.25 2.25 Stored-Program Concept — IMPORTANT

Instructions and data are both represented in binary and stored in memory.

Therefore a program can manipulate representations of other programs. Compilers and linkers are examples.

1.26 2.26 Compiler, Assembler, Linker, Loader — VERY IMPORTANT

Compiler: translates high-level source toward lower-level representation.

Assembler: converts assembly instructions into machine code/object modules.

Linker: combines object modules and libraries into an executable image, resolves labels, and patches external/location-dependent references.

Loader: loads the executable image into memory, sets up the virtual address space and stack, initializes registers, and transfers control to startup code.

1.27 2.27 Object Module — IMPORTANT

An object module can contain:

- header,
- text segment,
- static data segment,
- relocation information,
- symbol table,
- debug information.

1.28 2.28 Static vs Dynamic Linking — IMPORTANT

Static linking: library code is incorporated into the executable before execution.

Dynamic linking: library procedures are linked/loaded when needed.

Advantages of dynamic linking described in the source include avoiding executable bloat and allowing newer library versions to be picked up.

2 CHAPTER 3 — ARITHMETIC FOR COMPUTERS

The chapter covers integer addition/subtraction, multiplication, overflow, floating-point representation, IEEE-754, floating-point arithmetic, precision, and RISC-V floating-point instructions.

2.1 3.1 Integer Addition — **VERY IMPORTANT**

For signed two's-complement integers:

- adding numbers of opposite signs cannot cause signed overflow,
- adding two positives overflows if the result becomes negative,
- adding two negatives overflows if the result becomes positive.

Memory rule:

Same signs + wrong result sign = overflow.

2.2 3.2 Integer Subtraction — **VERY IMPORTANT**

Subtraction is performed as:

$$A - B = A + (-B).$$

The key idea is to negate the second operand using two's complement and then add.

The supplied handout notes that subtracting two same-sign operands does not overflow; different-sign subtraction requires checking the result sign.

2.3 3.3 Multiplication — **VERY IMPORTANT**

Binary multiplication follows long multiplication using partial products.

An n -bit by n -bit multiplication may require up to $2n$ bits of product.

2.4 3.4 Multiplication Hardware — **IMPORTANT**

The simplified hardware model uses repeated:

add + shift.

Optimized multiplier hardware can perform add/shift steps in parallel.

2.5 3.5 RISC-V Multiplication — **VERY IMPORTANT**

For RV64, the supplied material gives:

<code>mul</code>	lower 64 bits
<code>mulh</code>	upper 64 bits, signed $\times$ signed
<code>mulhu</code>	upper 64 bits, unsigned $\times$ unsigned
<code>mulhsu</code>	upper 64 bits, signed $\times$ unsigned

If full multiplication produces 128 bits:

upper 64 bits + lower 64 bits.

The high half can be used to determine whether the 64-bit result overflowed.

2.6 3.6 Integer Division — **KNOW** (resource limitation)

The supplied Chapter 3 slides list multiplication and division as chapter topics, but the retrieved course resources do not develop a detailed integer-division algorithm at the same level as multiplication. Do not invent an additional algorithm and attribute it to these resources.

2.7 3.7 Floating Point — **VERY IMPORTANT**

Floating point represents non-integral values and values with very large or very small magnitude.

Binary normalized form:

$$\pm 1.xxxxx \times 2^y.$$

The idea is analogous to decimal scientific notation.

2.8 3.8 Why Floating Point?

Fixed-size integers cannot conveniently represent all real values. Floating point separates a number into:

sign, significand, exponent.

This allows a large dynamic range.

2.9 3.9 IEEE-754 — **EXTREMELY IMPORTANT**

The supplied handout uses IEEE-754.

Two principal formats:

Single precision = 32 bits

Double precision = 64 bits.

Single precision

$$1 \text{ sign bit} + 8 \text{ exponent bits} + 23 \text{ fraction bits} = 32.$$

Double precision

$$1 \text{ sign bit} + 11 \text{ exponent bits} + 52 \text{ fraction bits} = 64.$$

Normalized numbers satisfy:

$$1.0 \leq |\text{significand}| < 2.0.$$

The leading 1 is implicit.

2.10 3.10 Sign Bit — **VERY IMPORTANT**

$$S = 0 \rightarrow \text{positive/non-negative}, \quad S = 1 \rightarrow \text{negative}.$$

2.11 3.11 Exponent Bias — **VERY IMPORTANT**

The stored exponent uses excess representation:

$$E_{\text{stored}} = E_{\text{actual}} + \text{Bias}.$$

Therefore:

$$E_{\text{actual}} = E_{\text{stored}} - \text{Bias}.$$

For the standard IEEE forms:

$$\text{Single bias} = 127, \quad \text{Double bias} = 1023.$$

Important source note: one supplied slide line appears to say 1203 for double bias, but the worked example uses 1023. The study value supported by the examples is 1023.

2.12 3.12 Fraction / Significand — VERY IMPORTANT

A normalized binary floating-point number is of the form:

$$1.\text{fraction} \times 2^{\text{exponent}}.$$

The leading 1 is hidden and not explicitly stored.

2.13 3.13 Floating-Point Precision — IMPORTANT

The supplied lecture gives approximately:

$$2^{-23}$$

for single precision and

$$2^{-52}$$

for double precision as relative-scale precision measures.

It associates these with roughly:

6 decimal digits

for single and

16 decimal digits

for double.

The important conceptual point is that floating-point representation has finite precision.

2.14 3.14 Special Floating-Point Values — VERY IMPORTANT

Infinity:

$$\text{exponent} = 111 \dots 111, \quad \text{fraction} = 000 \dots 000.$$

NaN:

$$\text{exponent} = 111 \dots 111, \quad \text{fraction} \neq 000 \dots 000.$$

NaN means “Not a Number” and can represent undefined or illegal results such as:

$$0.0/0.0.$$

2.15 3.15 Decimal to IEEE-754 — VERY IMPORTANT

The supplied handout gives:

1. convert the decimal number to binary,
2. normalize the binary number,
3. find the biased exponent,
4. determine the sign bit,
5. find the fraction,
6. encode according to the IEEE format.

2.16 3.16 IEEE-754 to Decimal — VERY IMPORTANT

The supplied handout gives:

1. convert the given hexadecimal/number to binary,
2. arrange the bits according to the IEEE format,
3. determine the sign,
4. determine the actual exponent from the biased exponent field,
5. convert the fraction to decimal,
6. calculate the final value.

2.17 3.17 Floating-Point Addition/Subtraction — **EXTREMELY IMPORTANT**

The supplied handout gives this process:

1. put both values in binary,
2. normalize both,
3. align the binary points by matching exponents,
4. add/subtract the significands,
5. normalize the result,
6. check overflow/underflow,
7. round and renormalize if necessary.

Why alignment? Because numbers with different exponents cannot be directly added until they have a common exponent.

2.18 3.18 Floating-Point Multiplication — **VERY IMPORTANT**

For:

$$A \times B$$

the supplied lecture gives:

1. add exponents,
2. multiply significands,
3. normalize,
4. check overflow/underflow,
5. round,
6. determine the sign.

For biased exponents:

$$E_{\text{result}} = E_A + E_B - \text{Bias}.$$

Sign:

$$(+) \times (+) = +, \quad (-) \times (-) = +, \quad (+) \times (-) = -.$$

2.19 3.19 Floating-Point Arithmetic Hardware — **IMPORTANT**

The supplied material says FP hardware commonly supports:

- addition,
- subtraction,
- multiplication,
- division,
- reciprocal,
- square root,
- FP-to-integer and integer-to-FP conversion.

Such operations may take multiple cycles and can be pipelined.

2.20 3.20 RISC-V Floating-Point Registers — **IMPORTANT**

The materials use separate floating-point registers:

$$f0, \dots, f31.$$

Single precision values use the lower 32 bits in the supplied description.

FP load/store instructions include:

`flw, fld, fsw, fsd`

2.21 3.21 RISC-V Floating-Point Instructions — **IMPORTANT**

Single precision:

```
fadd.s  
fsub.s  
fmul.s  
fdiv.s  
fsqrt.s
```

Double precision:

```
fadd.d  
fsub.d  
fmul.d  
fdiv.d  
fsqrt.d
```

Comparisons:

```
feq.s, flt.s, fle.s  
feq.d, flt.d, fle.d
```

The comparison result is 0 or 1 in an integer destination register in the supplied materials.

The handout also lists fused operations such as:

```
fmadd.s  
fmsub.s  
fnmadd.s  
fnmsub.s
```

and conversion/move instructions such as `fcvt.w.s`, `fmv.w.x`, and `fmv.x.w`.

2.22 3.22 Big Theory of Chapter 3 — **VERY IMPORTANT**

Bits themselves have no inherent meaning. Their interpretation depends on the instruction and representation being used.

A bit pattern may represent a signed integer, unsigned integer, floating-point encoding, or instruction field depending on context.

Computer number representations have finite range and finite precision. Programs therefore need to account for overflow, underflow, and approximation.

3 CHAPTER 4 — THE PROCESSOR

The conceptual flow of this chapter is:

Instruction → Datapath → Control → Single-cycle processor → Critical-path problem → Pipeline → Pipeline hazard

3.1 4.1 CPU Performance — **VERY IMPORTANT**

The supplied source identifies:

- instruction count,
- CPI (cycles per instruction),
- cycle time.

A core performance relationship is:

$$\boxed{CPU\ Time = Instruction\ Count \times CPI \times Cycle\ Time}$$

Instruction count is influenced by ISA and compiler; CPI and cycle time are influenced by CPU hardware.

3.2 4.2 Instruction Execution — **VERY IMPORTANT**

A simplified sequence is:

1. PC points to instruction memory,
2. instruction is fetched,
3. register operands are read,
4. ALU performs the required operation,
5. data memory is accessed for load/store instructions,
6. PC becomes target address or $PC + 4$.

3.3 4.3 Datapath — **VERY IMPORTANT**

A **datapath** consists of hardware elements that process data and addresses in the CPU.

Examples:

- registers,
- ALUs,
- multiplexers,
- memories.

3.4 4.4 Control Unit — **VERY IMPORTANT**

Control signals decide:

- which registers are read,
- which ALU operation is selected,
- which data source is selected,
- whether memory is read or written,
- whether a register is written,
- how the PC is updated.

3.5 4.5 Combinational vs Sequential Logic — **VERY IMPORTANT**

Combinational:

$$Output = f(Input).$$

Output depends only on current input.

Examples:

- AND gate,

- multiplexer,
- adder,
- ALU.

Sequential/state element: stores information and changes according to a clock.

Examples:

- registers,
- pipeline registers.

3.6 4.6 Multiplexer — IMPORTANT

A multiplexer chooses one of several inputs:

$$Y = S?I1 : I0.$$

Why needed? Different instruction types need different data sources, so control signals select the appropriate path.

3.7 4.7 Register — IMPORTANT

A register stores information in a circuit.

The supplied slides describe an edge-triggered register as updating when the clock changes from 0 to 1.

A register with write control updates only when:

$$\text{clock edge} \wedge \text{write} = 1.$$

3.8 4.8 Clocking Methodology — VERY IMPORTANT

Conceptually:

State element $\rightarrow$ Combinational logic $\rightarrow$ Next state element.

The longest combinational delay determines the minimum clock period.

Key phrase:

Critical path determines the minimum clock period.

3.9 4.9 Instruction Fetch — IMPORTANT

For 32-bit instructions in the supplied RISC-V datapath:

$$PC_{\text{next}} = PC + 4$$

for ordinary sequential instruction flow.

3.10 4.10 R-Type Datapath — VERY IMPORTANT

For an R-type instruction:

1. read two registers,
2. perform the arithmetic/logical operation in the ALU,
3. write the result into the destination register.

3.11 4.11 Load/Store Datapath — VERY IMPORTANT

For load/store:

1. read required register operands,
2. sign-extend the 12-bit offset,
3. calculate the effective address using the ALU,
4. load: read memory and update a register,
5. store: write register data to memory.

3.12 4.12 Branch Datapath — **VERY IMPORTANT**

For beq:

1. read two register operands,
2. compare operands,
3. subtract in the ALU,
4. inspect the Zero output,
5. sign-extend the branch displacement,
6. shift it left by one,
7. add it to the PC to form the target.

Why subtraction? If:

$$A - B = 0$$

then:

$$A = B.$$

3.13 4.13 Why Separate Instruction and Data Memory? — **VERY IMPORTANT**

If one memory were used simultaneously for instruction fetch and data access, the two operations could conflict.

The single-cycle datapath therefore uses separate instruction and data memories in the supplied architecture.

3.14 4.14 ALU Control — **EXTREMELY IMPORTANT**

The supplied lecture gives:

ALU Operation	ALU Control
AND	0000
OR	0001
ADD	0010
SUB	0110

High-level ALUOp:

ld	00
sd	00
beq	01
R-type	10

For R-type, the exact ALU operation is further selected from the instruction fields.

3.15 4.15 Why Does ld Use ADD in the ALU?

Because the processor calculates:

$$\text{effective address} = \text{base} + \text{offset}.$$

The ALU is being used for address calculation.

3.16 4.16 Single-Cycle Processor — **VERY IMPORTANT**

The first-cut datapath completes one instruction in one clock cycle.

Problem:

Every instruction must use the same clock period.

The clock must therefore be long enough for the slowest instruction.

3.17 4.17 Critical Path — **EXTREMELY IMPORTANT**IMPORTANT

The supplied lecture identifies load as the critical path:

Instruction memory → Register file → ALU → Data memory → Register file.

The longest delay determines the clock period.

A branch may need much less hardware work, but in a single-cycle design it still waits for the full clock period.

3.18 4.18 Pipelining — **EXTREMELY IMPORTANT**IMPORTANT

Pipelining means overlapping the execution of multiple instructions.

The core benefit is:

higher instruction throughput

rather than necessarily lower latency for one individual instruction.

3.19 4.19 Five Pipeline Stages — **EXTREMELY IMPORTANT**IMPORTANT

IF = Instruction Fetch

ID = Instruction Decode / Register Read

EX = Execute / Address Calculation

MEM = Memory Access

WB = Write Back

Memory trick:

IF ID EX MEM WB

3.20 4.20 Why Pipelining Improves Performance — **VERY IMPORTANT**

Without pipelining, instructions are processed sequentially.

With pipelining:

I_1 :	<i>IF</i>	<i>ID</i>	<i>EX</i>	<i>MEM</i>	<i>WB</i>	
I_2 :		<i>IF</i>	<i>ID</i>	<i>EX</i>	<i>MEM</i>	<i>WB</i>
I_3 :			<i>IF</i>	<i>ID</i>	<i>EX</i>	<i>MEM WB</i>

After the pipeline fills, multiple instructions are being processed simultaneously.

Thus:

throughput increases.

3.21 4.21 Why RISC-V Is Pipeline-Friendly — **IMPORTANT**

The supplied lecture identifies:

- all instructions are 32 bits,
- instruction fetching is easier,
- formats are few and regular,
- register fields are consistently positioned,
- load/store addressing fits naturally into the pipeline stages.

3.22 4.22 Pipeline Registers — **IMPORTANT**

Pipeline registers are placed between stages:

$$\text{IF} \rightarrow [\text{register}] \rightarrow \text{ID}$$

$$\text{ID} \rightarrow [\text{register}] \rightarrow \text{EX}$$

$$\text{EX} \rightarrow [\text{register}] \rightarrow \text{MEM}$$

$$\text{MEM} \rightarrow [\text{register}] \rightarrow \text{WB.}$$

They hold information produced during the previous cycle.

3.23 4.23 Pipeline Hazards — **EXTREMELY IMPORTANT**

A **hazard** is a situation that prevents the next instruction from starting normally in the next cycle.

Three types:

1. structural hazard,
2. data hazard,
3. control hazard.

3.24 4.24 Structural Hazard — **VERY IMPORTANT**

Occurs when two instructions require the same hardware resource at the same time.

Example: a single memory is needed both for instruction fetch and for a load/store data access.

Solution:

- separate instruction and data memories, or
- separate instruction and data caches.

3.25 4.25 Data Hazard — **EXTREMELY IMPORTANT**

Occurs when one instruction depends on a result produced by a previous instruction.

Example:

```
add x19, x0, x1
sub x2, x19, x3
```

The second instruction needs the result of the first.

3.26 4.26 Forwarding / Bypassing — **EXTREMELY IMPORTANT**

Instead of waiting for a result to be written back to the register file, it can be forwarded directly to where it is needed.

This reduces stalls.

3.27 4.27 Load-Use Hazard — **EXTREMELY IMPORTANT**

Example:

```
ld x1, 0(x2)
add x3, x1, x4
```

The loaded value is not available early enough for the immediately following instruction.

The key idea:

You cannot forward backward in time.

Therefore a stall may still be required.

3.28 4.28 Code Scheduling — IMPORTANT

Independent instructions can sometimes be moved between a load and its dependent instruction so that useful work occurs while waiting.

This reduces stalls.

3.29 4.29 Control Hazard — VERY IMPORTANT

A control hazard occurs when the next PC depends on a branch decision that is not yet known.

For a branch:

$$PC_{\text{next}} = \begin{cases} PC + 4, & \text{not taken} \\ \text{branch target}, & \text{taken} \end{cases}$$

Until the outcome is known, the pipeline may have to stall or otherwise handle the uncertainty.

4 CHAPTER 5 — LARGE AND FAST: MEMORY HIERARCHY

The central conceptual chain is:

program behavior → locality → cache → performance → virtual memory.

4.1 5.1 Principle of Locality — **EXTREMELY IMPORTANT**IMPORTANT

Programs usually do not access all memory uniformly. They repeatedly use a small region at a time.

Temporal locality:

Recently accessed items are likely to be accessed again soon.

Example: instructions in a loop.

Spatial locality:

Items near recently accessed items are likely to be accessed soon.

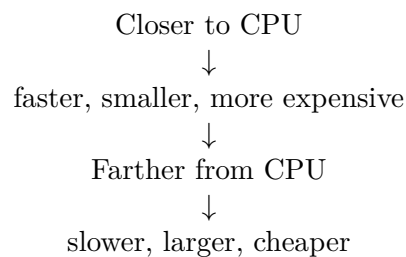
Example: array elements and sequential instruction fetch.

4.2 5.2 Memory Hierarchy — **EXTREMELY IMPORTANT**IMPORTANT

Different memory levels have different:

- speed,
- capacity,
- cost.

General trade-off:



This hierarchy exploits locality by keeping frequently needed information in faster levels.

4.3 5.3 SRAM vs DRAM — **IMPORTANT**

The supplied handout describes:

SRAM: uses multiple transistors per bit and does not need refresh while powered.

DRAM: uses a capacitor/transistor storage arrangement and requires periodic refresh because the stored charge leaks.

Conceptually:

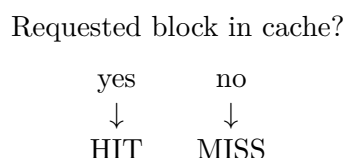
SRAM → faster, expensive, cache

DRAM → slower, cheaper, main memory.

4.4 5.4 Cache — **EXTREMELY IMPORTANT**IMPORTANT

A **cache** is a small, fast memory near the CPU used to exploit locality.

The basic decision is:



4.5 5.5 Block / Cache Line — **VERY IMPORTANT**

The minimum unit transferred between hierarchy levels is called a **block** or **cache line**. It may contain multiple words.

4.6 5.6 Hit, Miss, Hit Ratio, Miss Ratio — **EXTREMELY IMPORTANT**

Hit: requested data exists in the upper level.

Miss: requested data is absent from the upper level, so a lower level must supply it.

Hit ratio:

$$\text{Hit Ratio} = \frac{\text{Hits}}{\text{Total Accesses}}$$

Miss ratio:

$$\text{Miss Ratio} = \frac{\text{Misses}}{\text{Total Accesses}}$$

Also:

$$\text{Miss Ratio} = 1 - \text{Hit Ratio}.$$

Hit time: time to access the upper level and determine hit/miss.

Miss penalty: additional time needed to fetch the block from the lower level and supply it to the processor.

4.7 5.7 Cache Organizations — **EXTREMELY IMPORTANT**

Three organizations in the supplied material:

1. direct mapped,
2. fully associative,
3. set associative.

4.8 5.8 Direct-Mapped Cache — **EXTREMELY IMPORTANT**

Each memory block maps to exactly one cache location.

Mapping rule:

$$\text{Cache Index} = \text{Block Address} \bmod \text{Number of Cache Blocks}$$

Advantages:

- simple,
- fast lookup,
- simple hardware.

Disadvantage: different memory blocks can map to the same cache index and continually replace one another.

4.9 5.9 Tag and Valid Bit — **EXTREMELY IMPORTANT**

A cache entry needs to know:

1. which memory block is stored there,
2. whether the entry contains valid data.

Tag: identifies the memory block.

Valid bit:

1 → entry is valid

0 → entry is invalid.

4.10 5.10 Address Division — **EXTREMELY IMPORTANT**IMPORTANT

A direct-mapped address is divided into:

$$\boxed{TAG \mid INDEX \mid OFFSET}$$

Offset: identifies the byte/word inside the block.

Index: identifies which cache location to inspect.

Tag: identifies which memory block is currently there.

For a direct-mapped cache:

$$IndexBits = \log_2(\text{Number of Cache Blocks})$$

$$OffsetBits = \log_2(\text{BlockSize in bytes})$$

$$TagBits = AddressBits - IndexBits - OffsetBits.$$

4.11 5.11 How to Determine Hit or Miss — **EXTREMELY IMPORTANT**IMPORTANT

1. Determine tag, index, and offset bits.
2. Use the index to select the cache entry.
3. Check the valid bit.
4. If invalid, it is a miss.
5. If valid, compare tags.
6. Equal tags $\rightarrow$ hit.
7. Different tags $\rightarrow$ miss.

4.12 5.12 Fully Associative Cache — **IMPORTANT**

A block is not restricted to one specific cache index; placement is flexible across the cache.

Conceptually:

one block $\rightarrow$ many possible locations.

4.13 5.13 Set-Associative Cache — **IMPORTANT**

The cache is divided into sets.

A memory block maps to one set, but may occupy one of several positions within that set.

Conceptually:

direct mapped: one possible location

set associative: one set, several possible positions

fully associative: many/all possible positions.

4.14 5.14 Block Size — **VERY IMPORTANT**

Larger blocks exploit spatial locality.

Accessing address A increases the likelihood that nearby addresses $A + 1, A + 2, \dots$ will soon be needed.

However, larger blocks are not always better.

For fixed cache size:

larger block $\rightarrow$ fewer blocks $\rightarrow$ more competition/conflict.

Also:

- cache pollution increases,
- miss penalty increases.

4.15 5.15 Cache Pollution — IMPORTANT

If only a small part of a large fetched block is used, unused data occupies cache space and can evict useful data.

4.16 5.16 Early Restart — IMPORTANT

Instead of waiting for an entire block to arrive, return the requested word to the CPU as soon as it arrives.

4.17 5.17 Critical-Word-First — IMPORTANT

Fetch the requested word first, then bring the remaining words in the background.

4.18 5.18 What Happens During a Cache Miss? — VERY IMPORTANT

A cache miss stalls the processor while the required block is fetched from the next hierarchy level.
Instruction cache miss:

fetch stalls → bring instruction → restart fetch.

Data cache miss:

bring data → complete data access.

4.19 5.19 Write-Through — VERY IMPORTANT

On a write hit:

update cache + update memory.

Advantage: memory stays current.

Disadvantage: every write may incur slower memory activity.

A **write buffer** can hold pending memory writes so the CPU can continue.

4.20 5.20 Write-Back — VERY IMPORTANT

On a write hit:

update cache only.

The block is marked:

$\text{dirty} = 1$.

When a dirty block is replaced:

write block back to memory.

4.21 5.21 Write Allocation — IMPORTANT

For a write miss:

Write allocate:

fetch block into cache
then perform the write

No-write allocate / write around:

do not fetch the block
write directly to lower memory

The supplied handout explicitly associates write-around with write-through plus no-write-allocate.

4.22 5.22 AMAT — **EXTREMELY IMPORTANT**IMPORTANT

Average Memory Access Time:

$$AMAT = Hit\ Time + (Miss\ Rate \times Miss\ Penalty)$$

Interpretation:

- high miss rate hurts,
- high miss penalty hurts,
- low hit time matters too.

4.23 5.23 Multilevel Cache — **IMPORTANT**

Typical hierarchy:

$$CPU \rightarrow L1 \rightarrow L2 \rightarrow Main\ Memory.$$

Sometimes:

$$CPU \rightarrow L1 \rightarrow L2 \rightarrow L3 \rightarrow Main\ Memory.$$

The supplied material emphasizes:

- L1: small and very fast; focus on minimal hit time,
- L2: larger and slower; focus on low miss rate,
- deeper levels are progressively farther and slower.

4.24 5.24 Virtual Memory — **EXTREMELY IMPORTANT**IMPORTANT

Virtual memory uses main memory as a cache for secondary storage.

CPU and operating system cooperate to translate:

$$\text{Virtual Address} \rightarrow \text{Physical Address}.$$

A virtual-memory block is called a **page**.

A virtual-memory miss is called a **page fault**.

Each process gets its own virtual address space, supporting both large apparent memory and protection.

4.25 5.25 Why Virtual Memory? — **VERY IMPORTANT**

Two key conceptual reasons:

1. the program can use a large virtual address space while only some pages are resident in memory,
2. processes can have protected address spaces.

4.26 5.26 Pages — **IMPORTANT**

Virtual memory uses fixed-size pages. The supplied slide uses 4 KB as an example page size.

4.27 5.27 Page Fault — **EXTREMELY IMPORTANT**IMPORTANT

A page fault occurs when a required page is not currently in main memory.

The OS can:

1. locate the page on disk,
2. choose a page to replace,
3. write the victim page to disk if dirty,
4. read the required page into memory,
5. update the page table,
6. restart the faulting instruction.

A page fault is extremely expensive relative to a cache hit; the supplied lecture describes the cost as millions of clock cycles.

4.28 5.28 Page Table — **EXTREMELY IMPORTANT**IMPORTANT

A page table is an array of page table entries (PTEs), indexed by virtual page number.

A PTE may contain:

- physical page number,
- referenced/use information,
- dirty information,
- storage/swap information if the page is not currently resident.

4.29 5.29 Reference Bit — **IMPORTANT**

The reference/use bit helps track whether a page has been accessed recently.

The OS periodically clears it.

A page with reference bit 0 has not been used recently in the simplified model.

4.30 5.30 Dirty Bit — **VERY IMPORTANT**

A dirty bit indicates that a page has been modified.

If a dirty page is selected for replacement:

write it back to disk first.

Because disk writes are very expensive, write-back is preferred over write-through for this level.

4.31 5.31 TLB — **EXTREMELY IMPORTANT**IMPORTANT

TLB = **Translation Lookaside Buffer**.

It is a small fast cache of recent page-table entries inside the CPU.

Instead of:

virtual address → page-table memory lookup → actual memory access,

the TLB can often provide the translation immediately.

Memory trick:

TLB = cache for address translation

4.32 5.32 TLB Miss vs Page Fault — **EXTREMELY IMPORTANT**IMPORTANT

Do not confuse them.

TLB miss: The translation is not in the TLB. The page may still be in physical memory.

TLB miss → page table lookup → if resident, fill TLB and retry.

Page fault: The page itself is not in memory.

Page fault → OS fetches page from disk.

Thus:

TLB miss ≠ page fault

4.33 5.33 TLB Miss Handler — **IMPORTANT**

On a TLB miss:

1. detect the miss,
2. locate the PTE,
3. place the PTE in the TLB,
4. retry the instruction.

If the page is not resident, a page fault follows.

4.34 5.34 TLB and Cache Interaction — **IMPORTANT**

With a physically tagged cache, translation occurs before the physical-cache lookup.

Using virtual-address tags can create aliasing issues when different virtual addresses refer to the same physical address.

4.35 5.35 Memory Protection — **VERY IMPORTANT**

Virtual memory supports protection between tasks/processes.

The supplied material mentions:

- privileged/supervisor mode,
- privileged instructions,
- protected page tables and state,
- controlled system calls,
- RISC-V `ecall` as an example system-call exception.

4.36 5.36 Common Framework for Memory Hierarchy — **EXTREMELY IMPORTANT**

At every hierarchy level, ask four questions:

1. **Where can the block go?** — block placement.
2. **How do we find it?** — finding a block.
3. **What happens on a miss?** — replacement.
4. **What happens when data is written?** — write policy.

This is one of the most important unifying ideas of Chapter 5.

5 THE BIG CONCEPTUAL CONNECTIONS

5.1 Connection 1: Why do caches work? — **VERY IMPORTANT**

Programs exhibit:

temporal locality + spatial locality.

Therefore:

locality $\rightarrow$ cache $\rightarrow$ fewer slow-memory accesses $\rightarrow$ better performance.

5.2 Connection 2: Why does pipelining improve performance but create hazards? — **VERY IMPORTANT**

Pipelining overlaps instructions:

more parallel work $\rightarrow$ higher throughput.

But overlapping execution means multiple instructions simultaneously use:

- hardware resources,
- data,
- control flow.

Therefore:

pipelining $\rightarrow$ hazards $\rightarrow$ stalls / forwarding / scheduling.

5.3 Connection 3: Why is a single-cycle processor inefficient? — **VERY IMPORTANT**

Every instruction receives the same clock period, so the clock must accommodate the slowest instruction.

The load instruction's critical path is longest.

Therefore simple instructions waste time.

Pipelining breaks the work into stages and overlaps different instructions.

5.4 Connection 4: Why is RISC-V designed with regular instruction formats? — **VERY IMPORTANT**

Regular formats provide:

easier decoding + consistent register fields + simpler datapath/control + pipeline-friendly design.

5.5 Connection 5: Why does virtual memory behave like cache? — **VERY IMPORTANT**

Both follow the same broad pattern:

large, slow storage $\rightarrow$ small, fast storage.

Cache:

main memory $\leftrightarrow$ cache.

Virtual memory:

disk $\leftrightarrow$ main memory.

5.6 Connection 6: Cache and Virtual Memory Share the Same Framework — **EXTREMELY IMPORTANT**IMPORTANT

Both ask:

1. Where can the block go?
2. How do I find it?
3. What if it is not there?
4. What happens on replacement?
5. How are writes handled?

Therefore the same abstract ideas of:

placement, lookup, replacement, write policy
--

apply across memory hierarchy levels.

6 TOP THEORY TOPICS TO PRIORITIZE

Based on the faculty's emphasis on understanding theory, prioritize the following.

Chapter 2 — Must Understand

- ISA / instruction set
- RISC-V registers and conventions
- register vs memory vs immediate operands
- signed vs unsigned integers
- two's complement
- instruction formats
- R-format
- I-format
- S-format
- branch instructions
- signed vs unsigned comparison
- procedures
- `jal` / `jalr`
- stack and saved/temporary registers
- logical operations
- shifts
- synchronization / atomic operations

Chapter 3 — Must Understand

- signed overflow
- subtraction using two's complement
- multiplication theory
- `mul`, `mulh`, `mulhu`, `mulhsu`
- floating-point representation
- normalized numbers
- IEEE-754
- sign/exponent/fraction
- exponent bias
- precision
- infinity and NaN
- floating-point addition
- floating-point multiplication
- overflow/underflow

Chapter 4 — Must Understand

- datapath
- control
- combinational vs sequential elements
- clocking
- R-type datapath
- load/store datapath
- branch datapath
- ALU control
- critical path
- pipelining
- five pipeline stages

- pipeline registers
- structural hazards
- data hazards
- forwarding
- load-use hazard
- control hazards

Chapter 5 — Must Understand

- locality
- temporal vs spatial locality
- memory hierarchy
- cache
- hit/miss
- hit ratio/miss ratio
- hit time/miss penalty
- direct-mapped cache
- tag/index/offset
- cache hit/miss procedure
- block-size trade-offs
- write-through vs write-back
- write allocate vs no-write allocate
- AMAT
- multilevel cache
- virtual memory
- page/page fault
- page table
- TLB
- TLB miss vs page fault
- memory protection
- common memory-hierarchy framework

7 SELF-TEST: QUESTIONS YOU SHOULD BE ABLE TO ANSWER WITHOUT NOTES

Chapter 2

1. Why are registers faster than memory?
2. Why does RISC-V have multiple instruction formats instead of one?
3. Why are immediates useful?
4. Why does RISC-V use PC-relative branches?
5. What is the difference between `beq`, `blt`, and `bltu`?
6. Why do procedures need the stack?
7. Why are some registers temporary and others saved?
8. What problem does `lr.d/sc.d` solve?

Chapter 3

1. Why can adding two positive numbers cause overflow?
2. Why can't adding positive and negative numbers cause signed overflow?
3. Why is subtraction implemented through addition?
4. Why can multiplication require twice as many bits?
5. Why are there separate `mul` and `mulh` instructions?
6. Why do we need floating point?

7. Why is the leading 1 not stored?
8. Why is an exponent bias used?
9. Why must floating-point numbers be aligned before addition?
10. What is the difference between overflow and underflow?
11. What are infinity and NaN?

Chapter 4

1. What is the difference between datapath and control?
2. What is a combinational circuit?
3. What is a sequential circuit?
4. Why is the load instruction usually on the critical path?
5. Why does a single-cycle processor waste time?
6. Why does pipelining improve throughput?
7. Why doesn't pipelining automatically solve everything?
8. What is a structural hazard?
9. What is a data hazard?
10. What is forwarding?
11. Why can't forwarding always solve a load-use hazard?
12. What is a control hazard?

Chapter 5

1. Why does a cache work?
2. What is temporal locality?
3. What is spatial locality?
4. What is the difference between hit time and miss penalty?
5. Why can a direct-mapped cache have conflict problems?
6. What are tag, index, and offset?
7. Why does increasing block size initially help?
8. Why can increasing block size eventually hurt?
9. What is write-through?
10. What is write-back?
11. What is a dirty bit?
12. What is AMAT?
13. Why do modern processors use multiple cache levels?
14. What is virtual memory?
15. What is a page fault?
16. What is a TLB?
17. What is the difference between a TLB miss and a page fault?
18. How does virtual memory provide protection?

8 FINAL-NIGHT PRIORITY ORDER

A practical study order for the supplied material is:

1. **Chapter 5** — **Cache + Virtual Memory**
2. **Chapter 4** — **Datapath + Pipeline + Hazards**
3. **Chapter 3** — **IEEE-754 + Overflow + Floating-Point Arithmetic**
4. **Chapter 2** — **Instruction Formats + Procedures + Branches + Synchronization**

Within every chapter, master all items marked **VERY IMPORTANT** first.

9 THE FOUR CHAPTERS AS ONE BIG STORY

Chapter 2

ISA → instructions → registers/memory → branches/procedures → synchronization.

Chapter 3

binary representation → integer arithmetic → overflow → floating point → IEEE-754 → FP arithmetic.

Chapter 4

instruction execution → datapath → control → single-cycle processor → critical path → pipeline → hazards.

Chapter 5

program behavior → locality → memory hierarchy → cache → cache mapping → cache misses → AMAT → virtual memory.

Closing Reminder

The faculty suggestion is especially important: do not prepare by memorizing isolated definitions alone.

For each major topic, make sure you can answer:

1. **What is it?**
2. **Why is it needed?**
3. **How does it work?**
4. **What problem does it solve?**
5. **What trade-off or limitation does it have?**

If you can answer those questions for the **VERY IMPORTANT** topics in Chapters 2–5, you are studying the theory in the way your faculty's suggestion calls for.